

Module 10 — Database

Interview-relevant DB fundamentals for backend engineers: ACID, isolation levels & anomalies, indexing, query optimization, connection pooling (HikariCP), and schema migrations (Flyway/Liquibase).

10.1 Transactions & ACID

Core Concept

A transaction is an atomic unit of work. **ACID**: - **Atomicity** — all or nothing (rollback on failure). - **Consistency** — moves DB from one valid state to another (constraints hold). - **Isolation** — concurrent txns don't corrupt each other (degree set by isolation level). - **Durability** — committed data survives crashes (WAL/redo log, fsync).

Interview Q

Explain each ACID property with an example (e.g. bank transfer = atomicity + consistency). How is durability implemented? (*write-ahead log + fsync.*)

10.2 Isolation Levels & Anomalies

Why Interviewers Ask This

This is the crux of concurrency correctness and directly ties to `@Transactional(isolation=...)`.

Anomalies

- **Dirty read** — read uncommitted data.
- **Non-repeatable read** — re-reading a row returns different data (another txn updated + committed).
- **Phantom read** — re-running a range query returns new rows.
- **Lost update** — two txns overwrite each other's update.

Levels (SQL standard) — each prevents more

Level	Dirty	Non-repeatable	Phantom
READ UNCOMMITTED	possible		
READ COMMITTED	X		
REPEATABLE READ	X	X	(SQL std; MySQL InnoDB prevents via gap locks)
SERIALIZABLE	X	X	X

Defaults: **PostgreSQL & Oracle = READ COMMITTED**; **MySQL InnoDB = REPEATABLE READ**. Higher isolation = more locking/aborts = less concurrency. Many DBs use **MVCC** (readers don't block writers) + snapshot isolation.

ASCII

```
lower isolation ---- more concurrency, more anomalies ---->
READ UNCOMMITTED < READ COMMITTED < REPEATABLE READ < SERIALIZABLE
<---- fewer anomalies, more locking/contention <----
```

Interview Q / Follow-ups

- Define the anomalies; which level prevents which.
- Default isolation in Postgres vs MySQL.

- What is MVCC / snapshot isolation?
- How does isolation relate to optimistic/pessimistic locking (Module 5)?

10.3 Indexes

Core Concept

An index is a sorted data structure (usually a **B+ tree**) that speeds lookups from $O(n)$ scan to $O(\log n)$, at the cost of extra storage and slower writes (index maintenance).

Types & concepts

- **B-tree** — range + equality (default).
- **Hash** — equality only.
- **Composite** — multi-column; order matters (**leftmost-prefix rule**: an index on `(a,b,c)` helps `a`, `a,b`, `a,b,c`, not `b` alone).
- **Covering index** — index includes all columns a query needs → no table lookup.
- **Clustered** (table stored in index order, e.g. PK in InnoDB) vs **non-clustered/secondary** (points to row).
- **Selectivity** — high-cardinality columns index well; low-cardinality (e.g. boolean) often doesn't.

Common Mistakes

Indexing everything (write penalty, storage); function on indexed column (`WHERE lower(email)=` breaks index unless functional index); wrong composite order; not indexing FKs/join columns; ignoring `LIKE '%x'` (can't use B-tree).

Interview Q / Follow-ups

- How does a B+ tree index speed queries?
- Leftmost-prefix rule; composite index ordering.
- What is a covering index? Clustered vs non-clustered.
- When does an index hurt / not get used?

10.4 Query Optimization

Techniques

- **EXPLAIN/EXPLAIN ANALYZE** — read the plan: seq scan vs index scan, join type (nested loop/hash/merge), rows estimated vs actual, sort/filter.
- Add/fix indexes; rewrite to be **SARGable** (avoid functions on columns).
- Avoid `SELECT *`; select needed columns (enable covering indexes).
- Fix N+1 (Module 5). Paginate (keyset for deep pages).
- Batch writes; avoid per-row round trips.
- Watch for implicit type casts breaking index usage.
- Update statistics (**ANALYZE**) so the planner estimates well.

ASCII — reading a plan

```
EXPLAIN ANALYZE SELECT ...
-> Seq Scan on orders (cost.. rows=1M)  <-- red flag: full scan
   after index: Index Scan using idx_orders_user (rows=25)  <-- good
```

Interview Q

- How do you find and fix a slow query? (*EXPLAIN → index/rewrite → verify.*)

- What makes a predicate non-SARGable?
- Nested-loop vs hash vs merge join — when each?

10.5 Connection Pooling (HikariCP)

Why Interviewers Ask This

Connection pool exhaustion is a top production incident (Module 13). HikariCP is Spring Boot's default.

Core Concept

Opening a DB connection is expensive (TCP + auth). A **pool** keeps a bounded set of open connections and hands them out, blocking (up to `connectionTimeout`) when none are free.

Key settings

- `maximumPoolSize` — the cap (right-sizing is critical; often **fewer** than you think — see PostgreSQL's formula `connections ≈ ((core_count*2)+effective_spindles)`).
- `minimumIdle`, `connectionTimeout` (default 30s), `idleTimeout`, `maxLifetime` (< DB/infra idle timeout), `leakDetectionThreshold`.

Common Mistakes / Debugging

- Pool too small → threads wait/ `connectionTimeout` → latency spikes/timeouts.
- Pool too large → overwhelms DB (context switching, memory), worse throughput.
- **Holding connections during remote calls / long transactions** → exhaustion.
- Symptoms: `Connection is not available, request timed out`; monitor active vs idle, pending threads (Micrometer/HikariCP metrics).

Interview Q / Follow-ups

- Why use a connection pool?
- How do you size `maximumPoolSize`? Bigger isn't better — why?
- What causes pool exhaustion and how to diagnose it?
- `maxLifetime` vs DB idle timeout — why align them?

10.6 Schema Migrations: Flyway vs Liquibase

	Flyway	Liquibase
Format	versioned SQL (<code>V1__init.sql</code>)	XML/YAML/JSON/SQL changelogs
Style	SQL-first, simple	DB-agnostic abstraction
Rollback	forward-fix (undo in paid)	built-in rollback tags
Best for	teams comfortable with SQL	multi-DB, declarative

Both track applied migrations in a metadata table and run pending ones at startup; integrate with Spring Boot automatically.

Best Practices

Migrations are **immutable + forward-only** (never edit an applied script); backwards-compatible for zero-downtime deploys (expand/contract: add column → deploy → backfill → switch → drop later); separate DDL from data migration; test on a copy.

Interview Q

- Flyway vs Liquibase.
- How do you do a zero-downtime schema change? (*expand/contract, backward-compatible steps.*)
- Why never modify an applied migration?

Module 10 — One-Page Cheat Sheet

Topic	Key point
ACID	atomicity, consistency, isolation, durability (WAL)
Anomalies	dirty / non-repeatable / phantom / lost update
Isolation	RU<RC<RR<SER; Postgres=RC, MySQL=RR; MVCC snapshots
Index	B+ tree $O(\log n)$; leftmost-prefix; covering; clustered vs secondary
SARGable	no functions on indexed columns
EXPLAIN	seq scan bad; check estimates vs actual, join type
HikariCP	bounded pool; small is often better; align maxLifetime
Pool exhaustion	long tx / remote calls holding connections
Migrations	Flyway (SQL) vs Liquibase; immutable, forward-only, expand/contract

Module 10 — Top Interview Questions

1. Explain ACID; how is durability achieved?
2. Isolation levels and the anomalies each prevents; defaults per DB.
3. What is MVCC?
4. How do indexes work; leftmost-prefix; covering index.
5. When is an index not used / harmful?
6. How do you optimize a slow query (EXPLAIN)?
7. Why connection pooling; how to size HikariCP.
8. Causes and diagnosis of pool exhaustion.
9. Flyway vs Liquibase; zero-downtime migrations.
10. Clustered vs non-clustered index.

Module 10 — Common Mistakes

- Over-indexing (write cost) / not indexing join & FK columns.
- Functions on indexed columns (non-SARGable).
- Oversized connection pool overwhelming the DB.
- Long transactions holding connections.
- Editing an already-applied migration.

Module 10 — Mock Interview

1. "An API got slow after data grew." → **EXPLAIN ANALYZE**; likely seq scan → add proper (composite/covering) index; verify plan.

2. *"Connection is not available errors under load."* → pool exhaustion; check for long tx / remote calls inside tx, size the pool, add timeouts.
3. *"Two transfers double-spend a balance."* → lost update; raise isolation or use optimistic/pessimistic locking.
4. *"Add a NOT NULL column to a 500M-row table with zero downtime."* → expand/contract: add nullable + default, backfill in batches, then enforce constraint.
5. *"Why is MySQL's default isolation different from Postgres?"* → InnoDB REPEATABLE READ (gap locks prevent phantoms) vs Postgres READ COMMITTED; both use MVCC.

Next → Module 11: Observability.